Trabalho Prático 3
Serverless – Task 3

Vinicius Trindade Dias Abel
Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

viniciustda@ufmg.br

1. Implementação do Runtime Serverless

O runtime foi desenvolvido em Python para substituir a imagem de referência. A implementação seguiu a arquitetura de um loop de eventos:

- **Inicialização:** O runtime conecta ao Redis utilizando variáveis de ambiente para host e porta. Uma classe Context foi criada para simular o objeto de contexto, mantendo o dicionário env para persistência de dados.
- **Ciclo de Monitoramento:** O script verifica periodicamente (padrão de 5 segundos, mas configurável) a chave de entrada no Redis (metrics).
- **Carregamento Dinâmico:** Utilizando a biblioteca importlib, o runtime carrega o módulo do usuário localizado em /opt/usermodule.py. Foi implementada também a extensão para suporte a arquivos ZIP (descompactação e carregamento de main.py), como sugerido nos requisitos extras.
- **Execução e Persistência:** Ao detectar novos dados, a função handler é invocada passando o payload JSON e o objeto de contexto. O resultado (dicionário) é serializado e salvo na chave de saída do Redis.

2. Extensões e Flexibilidade

Além dos requisitos mínimos, o runtime foi estendido com as seguintes funcionalidades, configuráveis via variáveis de ambiente no Deployment:

- **MONITORING_PERIOD:** Permite ajustar a frequência de leitura do Redis.
- **HANDLER_NAME:** Permite definir um nome de função de entrada diferente de "handler".
- **Suporte a Funções Complexas (.ZIP):** O runtime foi preparado para executar funções compostas por múltiplos arquivos empacotados em formato ZIP. No arquivo deployment.yaml, foram incluídas as configurações para esta funcionalidade (variável de ambiente ZIP_FILE_PATH), que foram mantidas comentadas por padrão. Essa decisão foi tomada para manter a compatibilidade com a função simples da Task 1 (myModule.py) que está em execução.

  Para utilizar uma função complexa, o usuário deve:
  - Criar um ConfigMap contendo o arquivo ZIP binário.
  - Descomentar a variável ZIP_FILE_PATH no deployment.yaml.
  - Configurar o volume correspondente no Kubernetes.

O código do runtime prioriza a execução do ZIP caso a variável de ambiente esteja definida e o arquivo exista, garantindo flexibilidade entre funções simples e complexas sem precisar alterar o código do container.

3. Compatibilidade

O runtime garante total compatibilidade com a imagem original (lucasmsp/serverless:redis):

1. Respeita a mesma estrutura de diretórios (/opt/usermodule.py).
2. Implementa a mesma assinatura de função handler(input, context).
3. Utiliza as mesmas variáveis de ambiente para conexão com Redis (REDIS_HOST, REDIS_INPUT_KEY, etc.).
4. Mantém a persistência do atributo context.env entre chamadas, importante para o funcionamento da lógica de média móvel da Task 1.

Assim, a troca da imagem no deployment.yaml foi transparente para a função serverless existente.